

executed

July 11, 2026

1 Signal Transformations Toolkit

A self-contained notebook implementing the classic time-axis operations on a sampled signal x : **shift** by β , **scale** by α , **mirror**, and the combined transform $x(\alpha t + \beta)$ — with **separate continuous-time (CT)** and **discrete-time (DT)** versions, an **even/odd decomposition**, and **matplotlib subplot** visualisations.

Design notes - No `np.interp` is used. Interpolation between samples is hand-written and lives in **one replaceable line** (currently the mean of the two nearest samples, `0.5*(left+right)`), so you can swap it for linear/nearest whenever you like. - Every function works on **asymmetric axes** (e.g. `t` from -1 to 4) and zero-pads anything that falls outside the sampled support.

1.1 0. Imports & Core Interpolation

`interp_value` returns the signal value at a single argument `q`; `evaluate` vectorises it over an array of query points. This pair is the engine every transform below is built on.

```
[1]: import numpy as np
import matplotlib.pyplot as plt

def interp_value(axis, values, q, fill=0.0):
    """
    Value of a sampled signal at an arbitrary argument `q`.

    axis, values : sample locations (sorted ascending) and their values.
                  May be ASYMMETRIC; len(axis) == len(values).
    q             : argument at which the value is requested.
    fill          : returned when q is outside the sampled support (zero-pad).
    """
    axis = np.asarray(axis, dtype=float)
    values = np.asarray(values, dtype=float)

    # Outside the support -> zero-pad.
    if q < axis[0] or q > axis[-1]:
        return fill

    idx = np.searchsorted(axis, q)                # position of q within axis
```

```

# Exact hit on a sample.
if idx < len(axis) and np.isclose(axis[idx], q):
    return values[idx]
if idx > 0 and np.isclose(axis[idx - 1], q):
    return values[idx - 1]

# q lies strictly between axis[lo] and axis[hi] -> interpolate.
lo, hi = idx - 1, idx

# ---- INTERPOLATION RULE (change this ONE line to redefine interp) ----
return 0.5 * (values[lo] + values[hi])          # mean of nearest samples
# Linear: w = (q - axis[lo]) / (axis[hi] - axis[lo])
#         return (1 - w) * values[lo] + w * values[hi]
# -----

def evaluate(axis, values, query, fill=0.0):
    """Vectorised interp_value over an array of query points."""
    query = np.atleast_1d(np.asarray(query, dtype=float))
    return np.array([interp_value(axis, values, q, fill) for q in query])

```

1.2 1. Continuous-Time (CT) Transformations

Each function returns (t_{out}, y) sampled on the input grid by default. All are special cases of $y(t) = x(\alpha t + \beta)$.

function	maps to	effect
<code>ct_shift(t,x,)</code>	$x(t + \beta)$	$+\beta$ advances (shift left); use $-\beta$ to delay
<code>ct_scale(t,x,)</code>	$x(\alpha t)$	$ \alpha > 1$ compresses, $0 < \alpha < 1$ stretches
<code>ct_mirror(t,x)</code>	$x(-t)$	time reversal
<code>ct_affine(t,x, ,)</code>	$x(\alpha t + \beta)$	the general transform

```

[2]: def ct_shift(t, x, beta, t_out=None):
    """y(t) = x(t + beta). +beta advances (left); use -beta to delay."""
    t_out = t if t_out is None else np.asarray(t_out, float)
    return t_out, evaluate(t, x, t_out + beta)

def ct_scale(t, x, alpha, t_out=None):
    """y(t) = x(alpha*t). |alpha|>1 compresses, 0<|alpha|<1 stretches."""
    t_out = t if t_out is None else np.asarray(t_out, float)
    return t_out, evaluate(t, x, alpha * t_out)

```

```

def ct_mirror(t, x, t_out=None):
    """ $y(t) = x(-t)$  (time reversal)."""
    t_out = t if t_out is None else np.asarray(t_out, float)
    return t_out, evaluate(t, x, -t_out)

def ct_affine(t, x, alpha, beta, t_out=None):
    """ $y(t) = x(\alpha t + \beta)$  -- the general transform."""
    t_out = t if t_out is None else np.asarray(t_out, float)
    return t_out, evaluate(t, x, alpha * t_out + beta)

```

1.3 2. Discrete-Time (DT) Transformations

Same four operations on an integer index axis n . Integer shifts land exactly on samples; a non-integer argument (e.g. from a fractional α) falls back on the same interpolation rule from Section 0.

```

[3]: def dt_shift(n, x, k, n_out=None):
    """ $y[n] = x[n + k]$ .  $+k$  advances; use  $-k$  to delay."""
    n_out = n if n_out is None else np.asarray(n_out)
    return n_out, evaluate(n, x, np.asarray(n_out, float) + k)

def dt_scale(n, x, alpha, n_out=None):
    """ $y[n] = x[\alpha n]$ . Non-integer arguments use the interp rule."""
    n_out = n if n_out is None else np.asarray(n_out)
    return n_out, evaluate(n, x, alpha * np.asarray(n_out, float))

def dt_mirror(n, x, n_out=None):
    """ $y[n] = x[-n]$ ."""
    n_out = n if n_out is None else np.asarray(n_out)
    return n_out, evaluate(n, x, -np.asarray(n_out, float))

def dt_affine(n, x, alpha, beta, n_out=None):
    """ $y[n] = x[\alpha n + \beta]$ ."""
    n_out = n if n_out is None else np.asarray(n_out)
    return n_out, evaluate(n, x, alpha * np.asarray(n_out, float) + beta)

```

1.4 3. Even & Odd Decomposition

$$x_e(t) = \frac{1}{2}(x(t) + x(-t)), \quad x_o(t) = \frac{1}{2}(x(t) - x(-t))$$

Reflected values outside the support are treated as 0, so this runs on asymmetric axes too. By construction $x_e + x_o = x$ everywhere.

```
[4]: def even_odd(axis, x):
      """
      Return (even_part, odd_part) sampled on the SAME axis.
      even = 0.5*(x(t) + x(-t)) , odd = 0.5*(x(t) - x(-t))
      Works for CT and DT; reflected points outside support are zero.
      """
      axis = np.asarray(axis, float)
      x = np.asarray(x, float)
      x_ref = evaluate(axis, x, -axis)          # x(-t) via interpolation + zero-pad
      return 0.5 * (x + x_ref), 0.5 * (x - x_ref)
```

1.5 4. Example Signals (deliberately asymmetric)

- **CT:** a ramp→plateau→ramp pulse, nonzero on $[0, 3]$, sampled on the asymmetric axis $t \in [-1, 4]$.
- **DT:** a short triangle on the asymmetric index range $n \in [-2, 5]$.

```
[5]: # ----- Continuous-time signal on an asymmetric axis t in [-1, 4] -----
t = np.linspace(-1, 4, 501)

def _proto(tt):
    y = np.zeros_like(tt)
    up = (tt >= 0) & (tt < 1); y[up] = tt[up]          # ramp 0 -> 1
    top = (tt >= 1) & (tt < 2); y[top] = 1.0          # plateau
    dn = (tt >= 2) & (tt < 3); y[dn] = 3 - tt[dn]      # ramp 1 -> 0
    return y

x = _proto(t)

# ----- Discrete-time signal on an asymmetric range n in [-2, 5] -----
n = np.arange(-2, 6)
xn = np.array([0, 1, 2, 3, 2, 1, 0, 0], dtype=float)

print("CT: t in [%g, %g], %d samples" % (t[0], t[-1], t.size))
print("DT: n =", n)
print("DT: x[n] =", xn)
```

```
CT: t in [-1, 4], 501 samples
DT: n = [-2 -1  0  1  2  3  4  5]
DT: x[n] = [0.  1.  2.  3.  2.  1.  0.  0.]
```

1.6 5. Demonstrate Every Function

Quick numeric checks for both the CT and DT versions.

```
[6]: # ---- Continuous-time demos ----
_, y_sh = ct_shift(t, x, beta=1.0)          # x(t+1): pulse moves LEFT by 1
_, y_sc = ct_scale(t, x, alpha=2.0)        # x(2t): compressed by 2
```

```

_, y_mr = ct_mirror(t, x)                #  $x(-t)$ 
_, y_af = ct_affine(t, x, alpha=2.0, beta=1.0) #  $x(2t+1)$ 
xe, xo = even_odd(t, x)

sample_t = [-0.5, 0.5, 1.5, 2.5, 3.5]
print("t          :", sample_t)
print("x(t)       :", np.round(evaluate(t, x, sample_t), 3))
print("x(t+1)     :", np.round(evaluate(t, y_sh, sample_t), 3))
print("x(2t)      :", np.round(evaluate(t, y_sc, sample_t), 3))
print("x(-t)      :", np.round(evaluate(t, y_mr, sample_t), 3))
print("x(2t+1)    :", np.round(evaluate(t, y_af, sample_t), 3))
print("even+odd==x:", np.allclose(xe + xo, x))

```

```

t          : [-0.5, 0.5, 1.5, 2.5, 3.5]
x(t)       : [0.  0.5 1.  0.5 0. ]
x(t+1)     : [0.5 1.  0.5 0.  0. ]
x(2t)      : [0.  1.  0.  0.  0.]
x(-t)      : [0.5 0.  0.  0.  0. ]
x(2t+1)    : [0.  1.  0.  0.  0.]
even+odd==x: True

```

```

[7]: # ---- Discrete-time demos ----
_, yn_sh = dt_shift(n, xn, k=1)          #  $x[n+1]$ 
_, yn_sc = dt_scale(n, xn, alpha=2)      #  $x[2n]$  (downsample)
_, yn_mr = dt_mirror(n, xn)              #  $x[-n]$ 
_, yn_af = dt_affine(n, xn, alpha=2, beta=1) #  $x[2n+1]$ 
en, on = even_odd(n, xn)

print("n          :", n)
print("x[n]       :", xn)
print("x[n+1]     :", yn_sh)
print("x[2n]      :", yn_sc)
print("x[-n]      :", yn_mr)
print("x[2n+1]    :", yn_af)
print("even[n]    :", np.round(en, 2))
print("odd[n]     :", np.round(on, 2))
print("even+odd == x[n]:", np.allclose(en + on, xn))

```

```

n          : [-2 -1  0  1  2  3  4  5]
x[n]       : [0.  1.  2.  3.  2.  1.  0.  0.]
x[n+1]     : [1.  2.  3.  2.  1.  0.  0.  0.]
x[2n]      : [0.  0.  2.  2.  0.  0.  0.  0.]
x[-n]      : [2.  3.  2.  1.  0.  0.  0.  0.]
x[2n+1]    : [0.  1.  3.  1.  0.  0.  0.  0.]
even[n]    : [1.  2.  2.  2.  1.  0.5 0.  0. ]
odd[n]     : [-1. -1.  0.  1.  1.  0.5 0.  0. ]
even+odd == x[n]: True

```

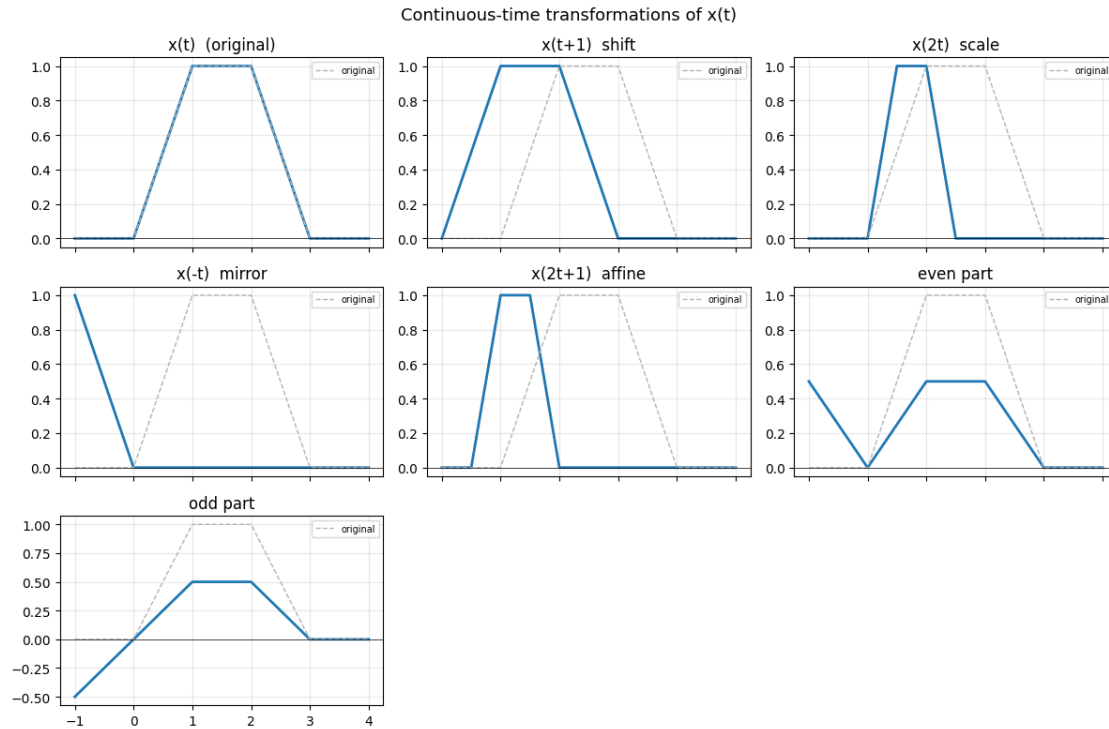
1.7 6. Plot Everything with Subplots

1.7.1 6a. Continuous-time

Each transform is drawn solid; the original $x(t)$ is overlaid dashed for reference.

```
[8]: panels = [
    ("x(t)  (original)", t, x),
    ("x(t+1)  shift",    t, y_sh),
    ("x(2t)   scale",    t, y_sc),
    ("x(-t)   mirror",   t, y_mr),
    ("x(2t+1)  affine",  t, y_af),
    ("even part",        t, xe),
    ("odd part",         t, xo),
]

fig, axs = plt.subplots(3, 3, figsize=(12, 8), sharex=True)
axs = axs.ravel()
for ax, (title, ta, ya) in zip(axs, panels):
    ax.plot(ta, ya, lw=2)
    ax.plot(t, x, "--", color="0.7", lw=1, label="original")
    ax.set_title(title); ax.grid(alpha=0.3); ax.axhline(0, color="k", lw=0.5)
    ax.legend(fontsize=7, loc="upper right")
for ax in axs[len(panels):]:
    ax.axis("off")
fig.suptitle("Continuous-time transformations of x(t)", fontsize=13)
fig.tight_layout()
plt.show()
```



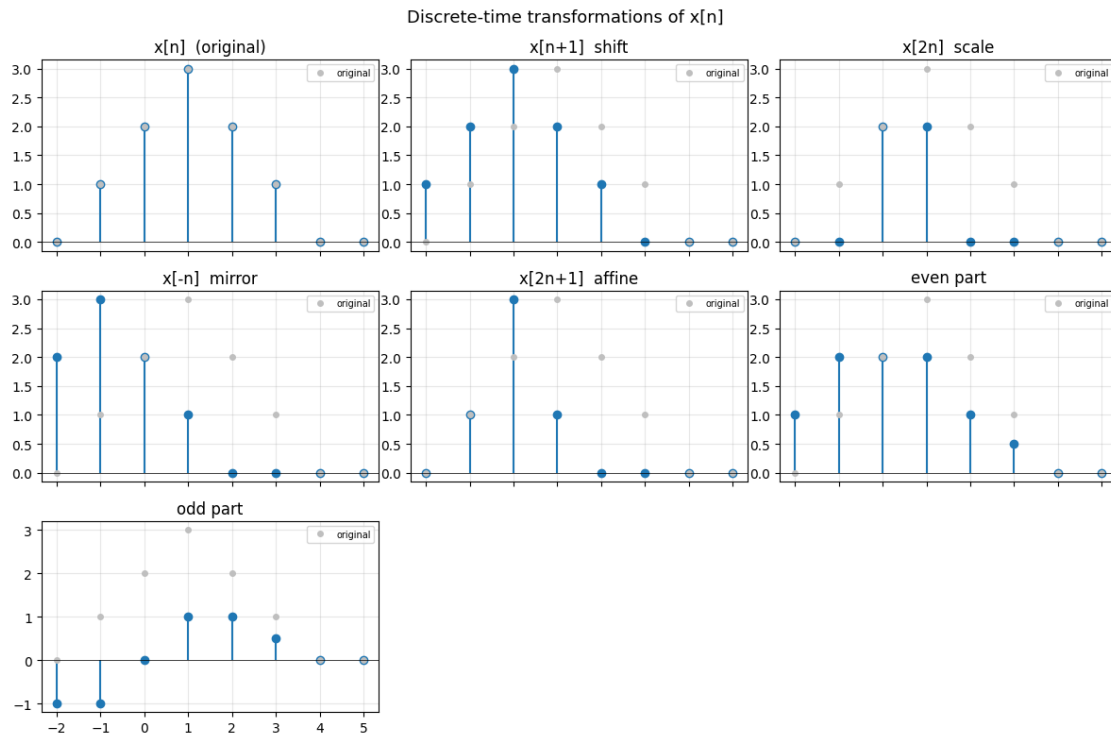
1.7.2 6b. Discrete-time

Stem plots of the same operations; the original $x[n]$ is shown as faint markers.

```
[9]: dpanels = [
    ("x[n] (original)", n, xn),
    ("x[n+1] shift", n, yn_sh),
    ("x[2n] scale", n, yn_sc),
    ("x[-n] mirror", n, yn_mr),
    ("x[2n+1] affine", n, yn_af),
    ("even part", n, en),
    ("odd part", n, on),
]

fig, axs = plt.subplots(3, 3, figsize=(12, 8), sharex=True)
axs = axs.ravel()
for ax, (title, na, ya) in zip(axs, dpanels):
    ax.stem(na, ya, basefmt=" ")
    ax.plot(n, xn, "o", color="0.75", ms=4, label="original")
    ax.set_title(title); ax.grid(alpha=0.3); ax.axhline(0, color="k", lw=0.5)
    ax.legend(fontsize=7, loc="upper right")
for ax in axs[len(dpanels):]:
    ax.axis("off")
fig.suptitle("Discrete-time transformations of  $x[n]$ ", fontsize=13)
```

```
fig.tight_layout()
plt.show()
```



Swapping the interpolation: edit the single marked line inside `interp_value` (Section 0). The default is the mean of the two nearest samples $0.5 * (\text{values}[\text{lo}] + \text{values}[\text{hi}])$; a linear alternative is shown just below it.